ENGPHYS 2E04:2024 Analog and Digital Circuits

EP2E04 – Design Project

Lab Section: L03-Wednesday
Instructor's Name: Dr. Jon Bradley

Written by: Ethan Chan
Lab Partner: Jocelyn Kuntsi

Date Lab Performed: Nov 27, 2024
Date of Submission: Dec 13, 2024

Introduction

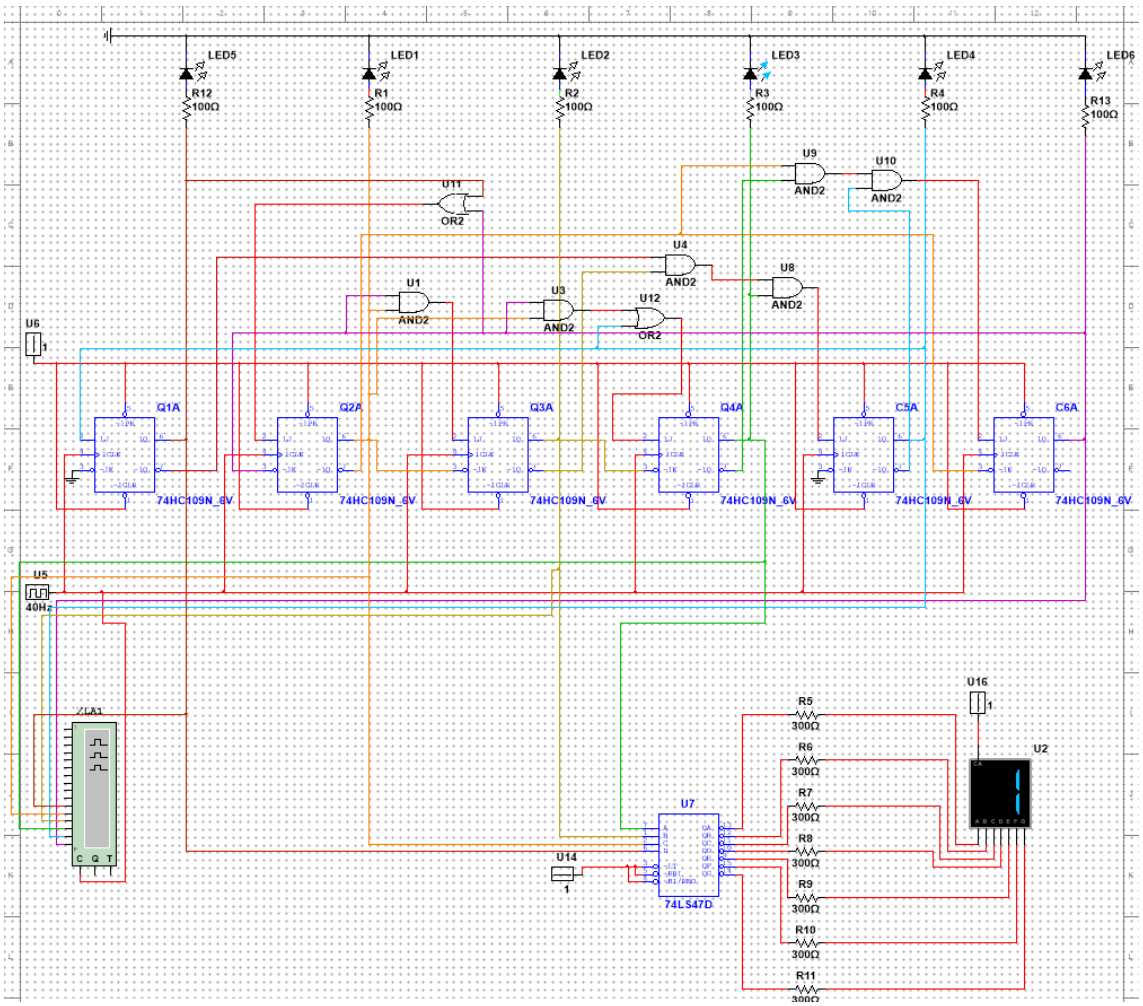

Figure 1: Multisim of Designed Circuit

Design Project Task using basic digital circuit tasked to build our own circuit that uses a 7-segment (7SD) to cycle through our student number using sequential logic design, For this sequential logic design, we found expression to input for J !K flip flop which output Q and !Q to design a finite state machine to cycle through our student number. In addition to the binary number decoder, we also need to incorporate other flip flops as internal "counter bits" in the finite state machine to distinguish between repeated digits and represent them as a state.

Design choices to optimize were the following:

1. Choosing what counter bits will do to make distinctions between states, and
2. Choosing logic gates that will correctly implement into the finite state machine to have an efficient design in Multisim and Physically

Student number for Individual Report: 400473109

Student number for Physical circuit: XXXXXXXX

Part 1: Analytical Solution

Binary Number for Student Number

The first step of the design project is to write your own student number digits as binary numbers. My number student number is 400473109, and it can be written in binary using 4 bits with “q” value corresponding to each.

Table 1: Student number written in binary numbers

Digit	q1 (8)	q2 (4)	q3 (2)	q4 (1)
4	0	1	0	0
0	0	0	0	0
0	0	0	0	0
4	0	1	0	0
7	0	1	1	1
3	0	0	1	1
1	0	0	0	1
0	0	0	0	0
9	1	0	0	1

Transition Table

Next, a transition table is used to determine the digits must transition to the next digit in the sequence and cycling to the beginning of the sequence. Counter variables c5 and c6 are introduced to account for repeats of digits that occur (0,4). As a design choice counters where forced to be 0 rather than X-“don’t care” and where there where repeats they would count up in binary (e.g. 1 = 01, 2= 10.)

Table 2: Transition table demonstrating cycling of each “q” bit value

Transition	q1	q2	q3	q4	c5	c6	Q1	Q2	Q3	Q4	C5	C6
4→0	0	1	0	0	0	0	0	0	0	0	0	0
0→0	0	0	0	0	0	0	0	0	0	0	0	**1**
0→4	0	0	0	0	0	**1**	0	1	0	0	0	**1**
4→7	0	1	0	0	0	**1**	0	1	1	1	0	0
7→3	0	1	1	1	0	0	0	0	1	1	0	0
3→1	0	0	1	1	0	0	0	0	0	1	0	0
1→0	0	0	0	1	0	0	0	0	0	0	**1**	0
0→9	0	0	0	0	**1**	0	1	0	0	1	0	0
9→4	1	0	0	1	0	0	0	1	0	0	0	0

Summary of Digits that occur: 4a, 0a, 0b, 4b, 7a, 3a, 1a, 0c, 9a ->4a

Excitation Table and Truth Table

Next, the excitation table (Table 3) for JK flip flops can be used to define what J and! K inputs for the (74HC109N) to change “q” to “Q” for each bit. This is shown in Table 4.

Table 3: Excitation table for JK flip flops.

q (Current State)	Q (Next State)	J	! K
0	0	0	X
0	1	1	X
1	0	X	0
1	1	X	1

Table 4: Truth table derived from the excitation table showing corresponding J and !K inputs.

q1	q2	q3	q4	c5	c6	Q1	Q2	Q3	Q4	C5	C6	J1	!K1	J2	!K2	J3	!K3	J4	!K4	J5	!K5	J6	!K6
0	1	0	**0**	0	0	0	0	0	**0**	0	0	0	X	X	0	0	X	0	X	0	X	0	X
0	0	0	**0**	0	0	0	0	0	**0**	0	1	0	X	0	X	0	X	0	X	0	X	1	X
0	0	0	**0**	0	1	0	1	0	**0**	0	1	0	X	1	X	0	X	0	X	0	X	X	1
0	1	0	**0**	0	1	0	1	1	**1**	0	0	0	X	X	1	1	X	**1**	**X**	0	X	X	0
0	1	1	**1**	0	0	0	0	1	**1**	0	0	0	X	X	0	X	1	X	1	0	X	0	X
0	0	1	**1**	0	0	0	0	0	**1**	0	0	0	X	0	X	X	0	X	1	0	X	0	X
0	0	0	**1**	0	0	0	0	0	**0**	1	0	0	X	0	X	0	X	X	0	1	X	0	X
0	0	0	**0**	1	0	1	0	0	**1**	0	0	1	X	0	X	0	X	1	X	X	0	0	X
1	0	0	**1**	0	0	0	1	0	**0**	0	0	X	0	1	X	0	X	X	0	0	X	0	X

Simplified Truth Table

The simplified truth table is created removing the Q (Next State), and in place just leaving the J and! K inputs to transition from q to Q. There were no further simplifications for the q1-q4 digit bits, and c5-c6 counter bits making for a six-bit truth table with J and! K inputs.

Table 5: Simplified truth table utilized for K-mapping.

q1	q2	q3	q4	c5	c6	J1	!K1	J2	!K2	J3	!K3	J4	!K4	J5	!K5	J6	!K6
0	1	0	0	0	0	0	X	X	0	0	X	0	X	0	X	0	X
0	0	0	0	0	0	0	X	0	X	0	X	0	X	0	X	1	X
0	0	0	0	0	1	0	X	1	X	0	X	0	X	0	X	X	1
0	1	0	0	0	1	0	X	X	1	1	X	1	X	0	X	X	0
0	1	1	1	0	0	0	X	X	0	X	1	X	1	0	X	0	X
0	0	1	1	0	0	0	X	0	X	X	0	X	1	0	X	0	X
0	0	0	1	0	0	0	X	0	X	0	X	X	0	1	X	0	X
0	0	0	0	1	0	1	X	0	X	0	X	1	X	X	0	0	X
1	0	0	1	0	0	X	0	1	X	0	X	X	0	0	X	0	X

General K-Map

The color changes along the columns were to generate the general K map with each colored column corresponding to one of the coordinates on the corresponding the six input K maps.

Table 6: General K-maps with the location of all significant values highlighted.

		c5=0	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00			X	X	00		X	X	X
c6=0	01		X	X		01	X	X	X	X
c6=0	11			X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00			X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

The process for K-mapping is as follows:

1. Fill input 1,0, X- "don't care" going down the column J1 to !K6 for Table 5 assigning the it to the corresponding color on Table 6.
2. Using SOP (Sum of Products): Making 2^n rectangles covering every 1 using as few rectangles as possible
    a. Find the constants within each rectangle, gets ANDed or multiplied, then each product term gets ORed or added (i.e. SUM of PRODUCTS).
3. Repeat for every J and! K value.

Table 7: J1 K-map

J1		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	0	0	X	X	00	1	X	X	X
c6=0	01	0	X	X	X	01	X	X	X	X
c6=0	11	0	0	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	0	0	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J1 = C5

Table 8: !K1 K-map

!K1		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	X	X	X	00	X	X	X	X
c6=0	01	X	X	X	0	01	X	X	X	X
c6=0	11	X	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K1 = 0

Table 8: J2 K-map

J2		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	0	X	X	X	00	0	X	X	X
c6=0	01	0	X	X	1	01	X	X	X	X
c6=0	11	0	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	1	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J2 = Q1 + C6

Table 9: !K2 K-map

!K2		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	0	X	X	00	X	X	X	X
c6=0	01	X	X	X	X	01	X	X	X	X
c6=0	11	X	0	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	1	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K2 = C6

Table 10: J3 K-map

J3		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	0	0	X	X	00	0	X	X	X
c6=0	01	0	X	X	0	01	X	X	X	X
c6=0	11	X	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	0	1	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J3 = (Q2)(C6)

Table 11: !K3 K-map

!K3		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	X	X	X	00	X	X	X	X
c6=0	01	X	X	X	X	01	X	X	X	X
c6=0	11	0	1	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K3 = Q2

Table 12: J4 K-map

J4		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	0	0	X	X	00	1	X	X	X
c6=0	01	X	X	X	X	01	X	X	X	X
c6=0	11	X	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	0	1	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J4 =C5 + (Q2)(C6)

Table 13: !K4 K-map

!K4		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	X	X	X	00	X	X	X	X
c6=0	01	0	X	X	0	01	X	X	X	X
c6=0	11	1	1	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K4 = Q3

Table 14: J5 K-map

J5		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	0	0	X	X	00	X	X	X	X
c6=0	01	1	X	X	0	01	X	X	X	X
c6=0	11	0	0	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	0	0	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J5 = (!Q1)(!Q3)(Q4)

Table 15: !K5 K-map

!K5		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	X	X	X	00	0	X	X	X
c6=0	01	X	X	X	X	01	X	X	X	X
c6=0	11	X	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K5=0

Table 16: J6 K-map

J6		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	1	0	X	X	00	0	X	X	X
c6=0	01	0	X	X	0	01	X	X	X	X
c6=0	11	0	0	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	X	X	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement J6 = (!Q2)(!Q4)(!C5)

Table 17: !K6 K-map

!K6		**c5=0**	**c5=0**	**c5=0**	**c5=0**		**c5=1**	**c5=1**	**c5=1**	**c5=1**
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=0	00	X	X	X	X	00	X	X	X	X
c6=0	01	X	X	X	X	01	X	X	X	X
c6=0	11	X	X	X	X	11	X	X	X	X
c6=0	10	X	X	X	X	10	X	X	X	X
	q1q2 / q3q4	00	01	11	10	q1q2 / q3q4	00	01	11	10
c6=1	00	1	0	X	X	00	X	X	X	X
c6=1	01	X	X	X	X	01	X	X	X	X
c6=1	11	X	X	X	X	11	X	X	X	X
c6=1	10	X	X	X	X	10	X	X	X	X

SOP Statement !K6 =!Q2

Boolean Expression Summary Table

Table 18: Boolean Expression Summary Table

Variable	Input Boolean Expression
J1	C5
!K1	0
J2	Q1 + C6
!K2	C6
J3	(Q2)(C6)
!K3	Q2
J4	C5 + (Q2)(C6)
!K4	Q3
J5	(!Q1)(!Q3)(Q4)
!K5	0
J6	(!Q2)(!Q4)(!C5)
!K6	!Q2

Post K-Mapping Simplification and Design Choices

Due to my student number, and its derived expressions, I could not make any further simplification. My student number (400473109) has 9 > 7 so it does require inputs Q1-Q4 to represent all my digits. Furthermore, my number digits repeat 0 three times as triplet, requiring 2 counter bits C5 and C6 to represent each unique state.

Upon further inspection, none of the Qs remained constant throughout the cycle, so none of the six required J !K flip flops could be removed. While POS (Product of Sums) could be explored to possibly create simpler expressions involving more Sums (+) OR expressions, there were more ones (1) compared to zeros (0) on the K-maps.

The logical expressions could not be reduced further with Boolean algebra simplifications from the absorption laws. De Morgans Laws could have been a consideration, for J5 and J6 NOT (!) operator and triple product.

I chose to make the following design choices easier: 1) following the K-map Boolean expression use 6 x AND gates with exception of the re-use of (Q2)(C6) which did not impact the number of AND chips requiring 2. For my organization sake tracing the Table 18 outputs, and working Multisim I used AND gate (Q2)(C6) twice. Although the choice would impact he wring of the physical circuit requiring another AND gate (Q2)(C6), instead of just taking its output to an OR gate for J4 with expression C5 + (Q2)(C6). 2) With 2 x OR gates requiring only 1 OR chip, 3) Apply the NOT operator we could use -Q from the flip flops instead of using a NOT gate for Qs.

The physical circuit would be composed of:

- 6 x JK flip flops Chips
- 6 -1(Re-use (Q2)(C6)) = 5 x AND gates, 2 AND Chips
- 2 x OR gates, 1 OR Chip
- 9 x Total Chips

Notice the Re-use of U1 AND (Q2)(C6) reducing the number of wires need for a Multisim representation the circuit which is not much more work tracing down the K-map expressions on Table-18, is a possible design choice. *Although this would be especially important impacting physical representation, checking two new inputs for a new AND gate (Q2)(C6) and one new output for U12 OR.*

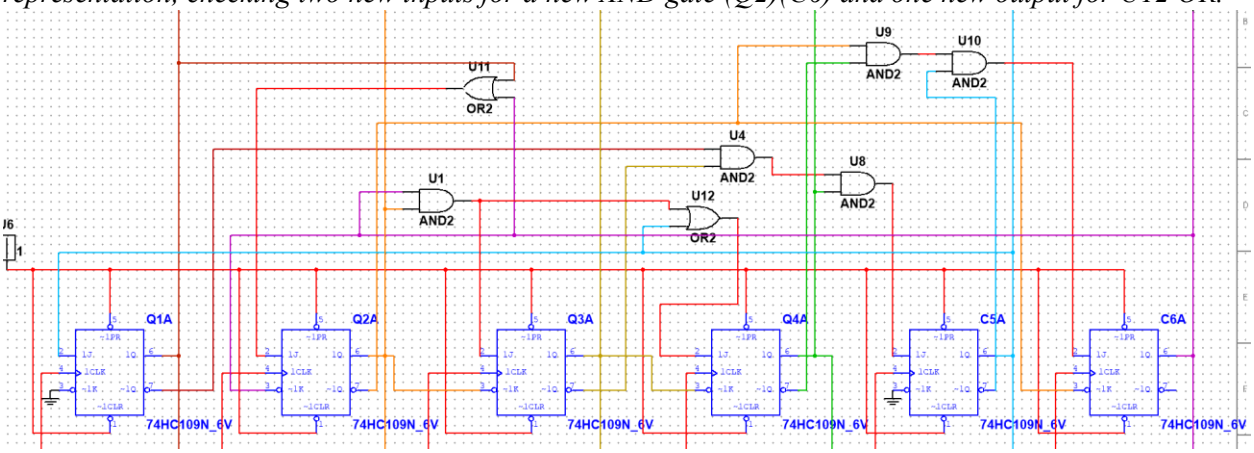

Figure 2: Multisim with Simplification of Designed Circuit re-use U1 AND (Q2)(C6)

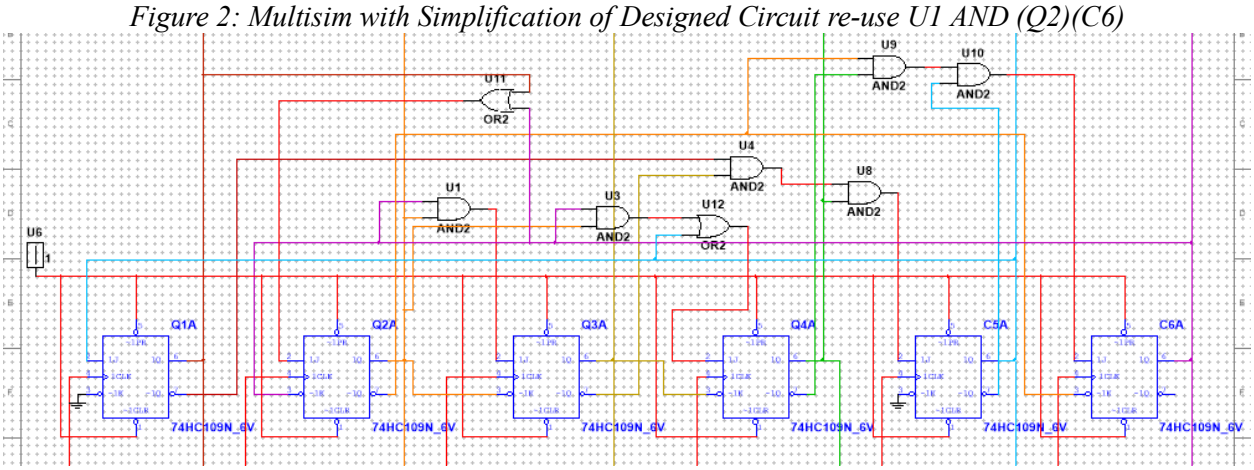

Figure 3: Multisim with Simplification of Designed Circuit without Re-use of U1 AN, instead introducing U3 AND (Q2)(C6)

Part 2: Multisim

Based on Table 18, the circuit was created using 6 x (74HC109N) JK flip flops, as well as generic 2-Input gates being 6 x AND, and 2 x OR gates. This would make for 6 x flip flops and 2 x (74HC00N) AND gate chips, and 1 x (74HC32N) OR gate chips, a total of 9 chips in a physical implementation. A decoder chip was used to decode Q1, Q2, Q3 and Q4 to display the digit they represent on the 7-segment display. For visual organization the wires were colour coded in Multisim using ROYGBV as Q and Q-outputs. Clock, NOT Preset, and NOT Clear remained red, and were all set to high, as well those wires set to the Decoder and 7-segment display.

Table 19: Colour coding on the wires in Multisim Implementation.

Function	Colour
Maintenance, Preset, Clear, Clock	Red
Q1	Red
Q2	Orange
Q3	Yellow
Q4	Green
C5	Blue
C6	Violet

The circuit was designed in five parts areas in mind: 1) Maintenance Clock, Preset, and Clear with JK flip flops 2) Logic Gates 3) LED outputs 4) Timming Diagram / Oscilloscope display 5) Decoder and 7-segment Display.

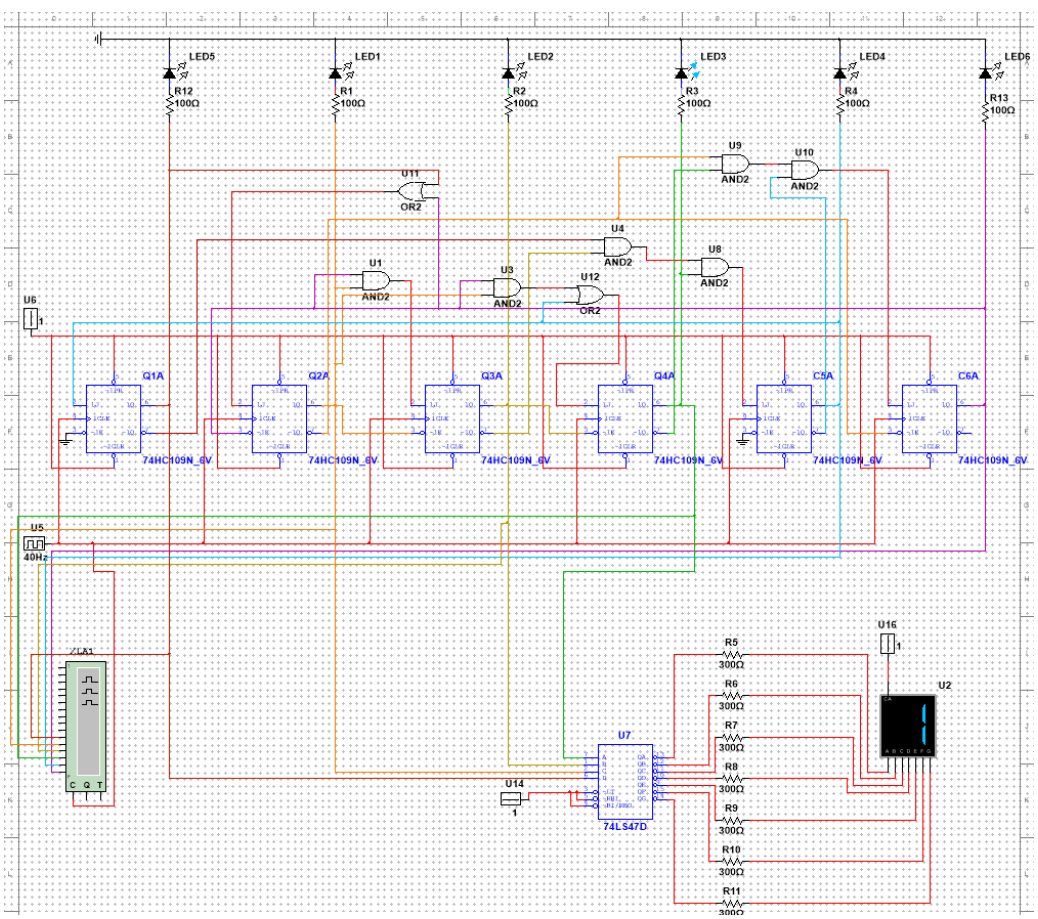

Figure 4: Multisim of Designed Circuit

Area 1) Maintenance Clock, Preset, and Clear with JK flip flops contained JK flip flops Q1 to C6, with the output of each one following the ROYGBV color coding scheme from Table 19 with the Qs and Cs outputs leading to their respective to LED, Timming Diagram, Decoder, and logic gate inputs.

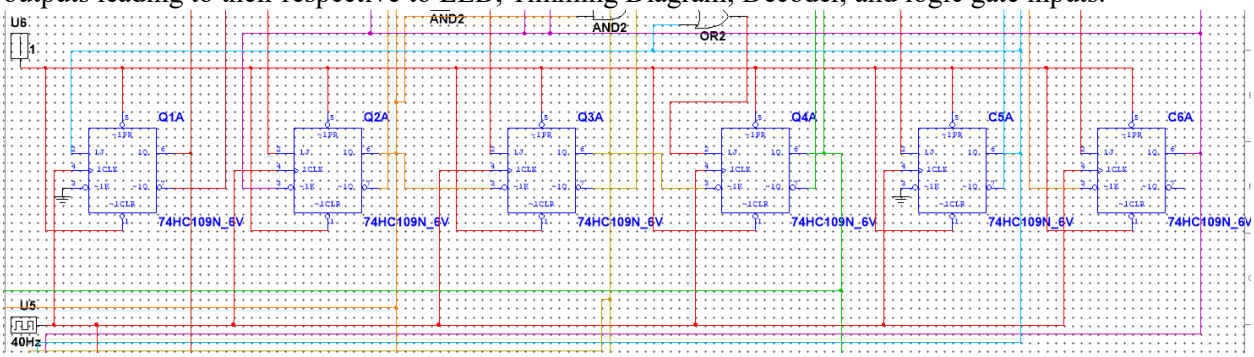

Figure 5: Area 1) Maintenance Clock, Preset, and Clear with JK flip flops

Area 2) Logic Gates based off logical expressions in Table 18, logical gates were used to create inputs for Js and!Ks from the outputs Qs, and -Qs from the flip flops. Many of these logic gates shared inputs with each other, which made wiring look like a mess, but it still adhered to the ROYGBV color scheme in Table 19 to make it easier to trace. To ensure correct wiring I went down Table 18 tracing Js and !Ks one input at time.

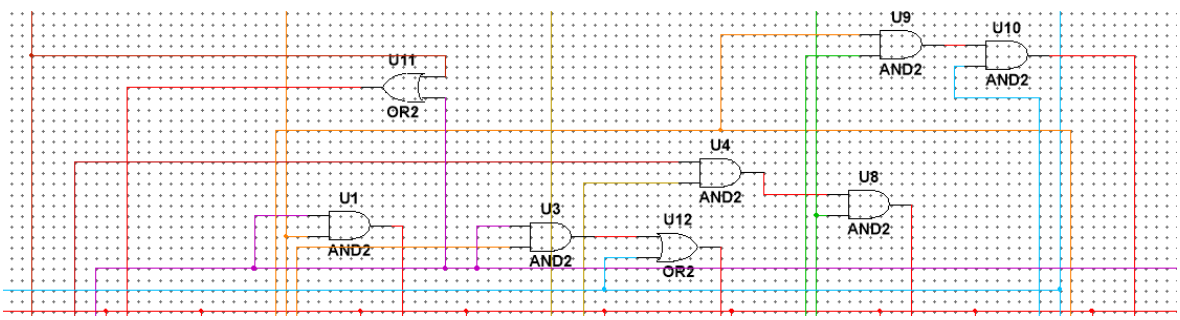

Figure 6: Area 2) Logic Gates

Area 3) LED outputs were used to ensure that Qs and Cs were outputting the correct state for digits as binary and correct counter ON (1) and OFF (2). The LEDs were to track the if the binary representation of digits was the same as the decoder to the 7-segment display, to ensure that the wiring for the decoder and display was correct. This output is also traceable on a timing diagram.

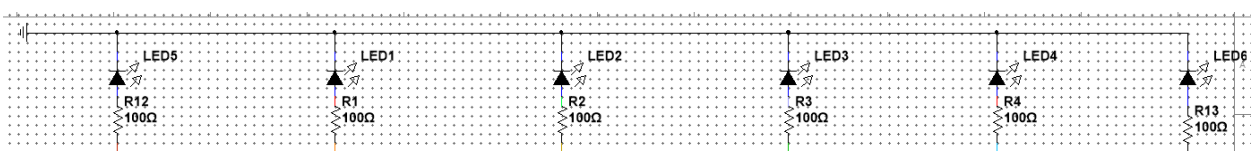

Figure 7: Area 2) Logic Gates

Area 4) Timming Diagram / Oscilloscope display is producing the timing diagram tracking the each Qs and Cs state update on the clock cycle's rising edge. In a lab setting the timing diagram and clock cycle can be produced and read off an oscilloscope.

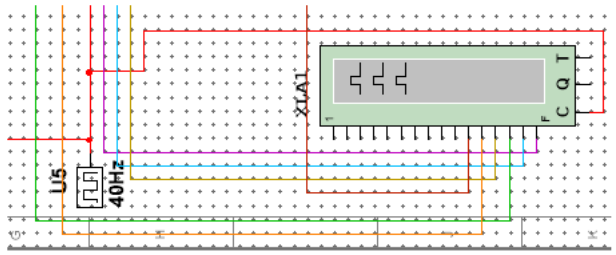

Figure 8: Area 4) Timming Diagram / Oscilloscope display

Lastly Area 5) Decoder and 7-segment Display contained the decoder chip (74LS47D) and the 7-segment display. Only Q1-Q4 were used as inputs for the decoder as the represent the digits. The set of resistors is used to lower the current passing into the 7-segment display in order to protect it from excess current.

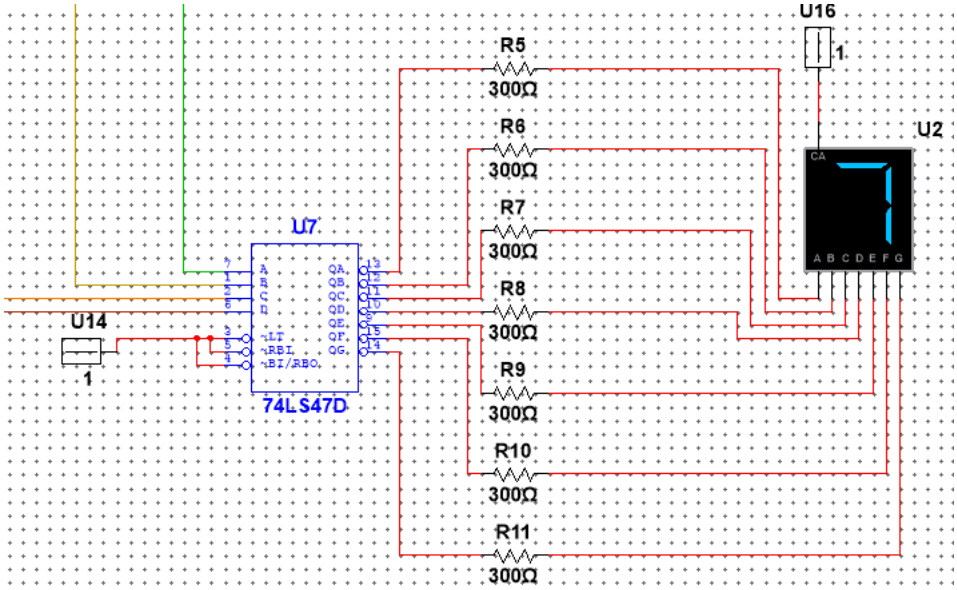

Figure 9: Area 5) Decoder and 7-segment Display

Video of Design Project Circuit

Below is a figure of the entire circuit working cycling through my student number 400473109.

My circuit was not chosen for the physical lab and is not part of the physical lab video.

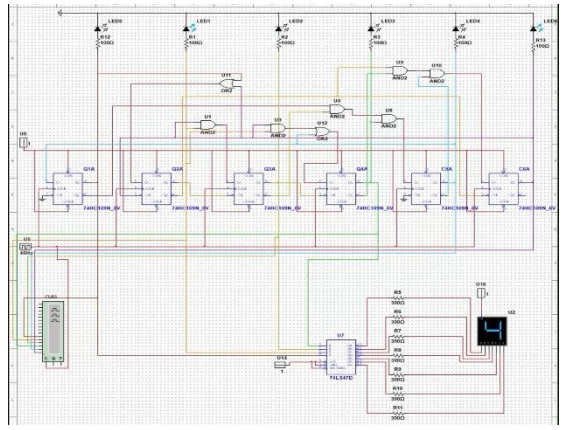

Figure 10: Video of Design Project circuit cycling through 400473109

Multisim Timming Diagram

Figure 11: Multisim Timming Diagram of Circuit for 400473109

As can be seen on the timing diagram displaying two cycles, the circuit goes through the student number in the following order (Q1, Q2, Q3,Q4,C5,C6):

((010000) → (000000) → (000001) → (010001) → (011100) →(001100) →(000100) →(000010) →(100100)) repeat.

Which represents: (4a→0a →0b →4b →7a →3a →1a →0c →9a) repeat.

We can visualize and label these states on the timing diagram. The white rectangles on the left represent contain each binary state and counter for each digit. We can see the same cycle after 9.

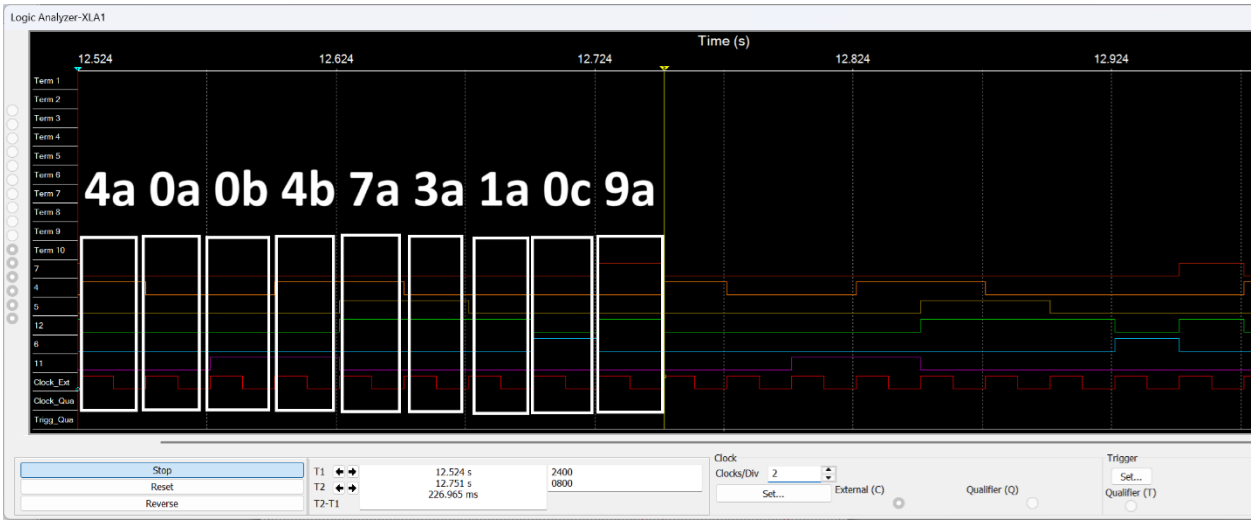

Figure 12: Multisim Timming Diagram of Circuit for 400473109 with marked states

Based off the Multisim simulation, 7-segment display and respective timing diagram, we can conclude the analytical circuit solution, and logic design were correct. This can be viewed again in the *Figure 7* Video. Due to lack of simplifications apart from reuse of AND gate U1 (Q2)(C6) as design choice discussed in the analysis the circuit was not physically built.

In place My lab partner Jocelyn Kuntsi, and I built her student number for physical circuit: XXXXXXX which had four JK flip flop chips, opposed to my six JK flip flop chips. Jocelyn's circuit had a total of 8 chips, opposed to my 9 total chips.